Design e Arquitectura de Software

2022/2023

Lab. Classes - 7

Java web services with Spring Boot

The following exercises are intended to be based on the web service tutorial project that you created and ran in the last lab.
The objective of this lesson will be to implement resource access with GET and LIST standard methods, and also field masks. Other standard methods will be addressed in later classes.

1 – Start by creating a department Resource. This resource should include a name and size (in addition to the unique identifier). Make sure you add public getters and setters for this field (syntax getXXX and setXXX where XXX is the field name).
Statically initialize three resources, eg.:

```
departments.add(new Department(1,"Sales",10));
departments.add(new Department(2,"Maintenance",20));
departments.add(new Department(3,"Admin",2));
```

We now wish to implement the GET and LIST standard methods. These correspond to the following http requests, respectively

```
<host>/departments/
<host>/departments/<dept id>
```

To achieve this, you should create a DepartmentController class, which will handle these requests, using the following structure.:

```
@RestController
class DepartmentController{
…
@GetMapping("/departments")
public List<Department> listDepartments() {…}
…
@GetMapping("/departments/{id}")
public Department getDepartment(@PathVariable("id") Integer id) {…}
```

These methods produce a JSON representation of the resource(s). By default, Spring uses the Jackson library to automatically obtain the correct JSON representation for any POJO you return as a response (having the proper setter/getter methods defined is necessary for this to happen).
At this point, you should be able to open your browser and check if things are working properly. You can do this by accessing the urls

- **localhost:8080\departments** - for obtaining a JSON listing all departments. This is the LIST standard method.

- **localhost:8080\departments\1** - for obtaining a JSON description of department 1. This is the GET standard method.

In order to make sure that method behavior is correct, we also need to make sure we are returning the appropriate response in all occasions. For example, the GET method should return a 404 error if the resource does not exist.
There are several ways to accomplish this. One way that is suitable for creating service responses is changing the return type of the method to a `ResponseEntity`. In our example, instead of returning a `Department` object, we would instead return a `ResponseEntity<Department>` object. The `ResponseEntity` class offers methods that allow customization of all aspects of the http response, including body, headers and status code. As an example,

new ResponseEntity<>(HttpStatus.***NOT_FOUND***)

Generates a response object corresponding to empty body and NOT FOUND status, while:

new ResponseEntity<>(dept,HttpStatus.***OK***)

creates an object representing a response with body obtained from object dept and status code HTTP OK. For further details, consult other resources such as the Spring documentation or https://www.baeldung.com/spring-response-entity.

There are several options available for Spring exception and error handling. You can check a detailed overview including other alternative approaches and ways to define your own custom error messages at this link (https://spring.io/blog/2013/11/01/exception-handling-in-spring-mvc).

Make these changes and check that the appropriate response is provided when GET-ing an inexistent resource (e.g using the debug mode of the browser to check HTTP response).

2 – Create now a `Person` class (with name and address). Add a new manager field (which is, not surprisingly, a Person) to the Department resource. Initialize this information appropriately, e.g.

```
departments.get(0).setManager(new Person(1,"John","Lisbon"));
// etc
```

Confirm that the GET and LIST operations correctly return the new information you just added.

3 – Now add to each Department a List of employees (also `Persons`). After properly initializing this info:

```
departments.get(0).getEmployees().add(new Person(2,"Anna","Coimbra"));
departments.get(0).getEmployees().add(new Person(3,"Charles","Coimbra"));
```

Confirm the employee information is also properly retrieved.

3 – Field masks are used to select the fields that should appear in a response. For example, http://localhost:8080/departments/1?fieldMask=name

Would return only the name (and id) of the requested department, while.

http://localhost:8080/departments/1?fieldMask=name&fieldMask=size

Would return both name, size (and id).
To do this we cannot rely directly on Jackson's JSON parsing, as it will always parse all fields. One approach to do this is to first create a map that includes all the desired information, which is then processed normally to obtain the desired JSON object. If a `LinkedHashMap` is used, then the fields will appear in the same order they are inserted (otherwise, they will be randomly ordered). As an example, the following returns a JSON object including only the id, name and manager of a department, in that order.

```
@GetMapping("")
Map<String,Object> hello() {
    Department d=departments.get(0);
    Map<String,Object> dynamic=new LinkedHashMap<>();
    dynamic.put("id",d.getid());
    dynamic.put("name",d.getName());
    dynamic.put("manager",d.getManager());
    return dynamic;
}
```

You can find other approaches for addressing dynamic JSON generation here (https://www.baeldung.com/jackson-mapping-dynamic-object).

The field masks are passed as request parameters (see previous Exercise sheet), more specifically, as a multivalued request parameter. They can be acessed by specifying a parameter in the handling methods such as be retrieved like this:

```
@RequestParam(value = "field", defaultValue = "*") List<String> fields
```

Notice how we are defining a default value of "*" (meaning all the fields) and that the parameter is specified as a List of String, rather than a single string, to allow the possibility of multiple values for the "field" parameter.

Make the necessary changes to allow field selection in GET and LIST methods. Check that it is working properly in your browser.

4 – Almost done. Field masks can refer to structured and repeated information (e.g. manager.name or employees.*.name). Implement these capabilities.